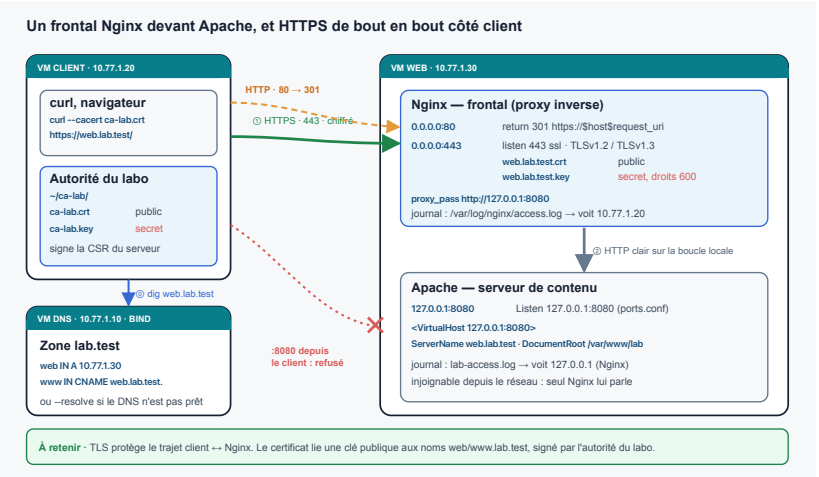
Web : Apache, Nginx et HTTPS

Hôtes virtuels, proxy inverse, certificats et TLS



Document
BTS SIO 2e année
SISR

Portée
Transversale

Thème
Service web du laboratoire

Objectifs

- **Publier** un site par un hôte virtuel Apache, puis **placer** Nginx en frontal.
- **Faire signer** un certificat par l'autorité du laboratoire et **passer** le site en HTTPS.
- **Prouver** ce que HTTPS garantit, et ce qu'il ne garantit pas.

Prérequis : VM du laboratoire, DNS `lab.test`, chiffrement asymétrique et signature (P01).

Que se passe-t-il derrière `https://web.lab.test/` ?

Étape	Protocole · port	Ce qui est obtenu	Preuve
Résolution	DNS · 53	L'adresse `10.77.1.30`	`dig @10.77.1.10 web.lab.test`
Connexion	TCP · 443	Un canal vers le serveur	`ss -ltnp` , `curl -v`
Négociation	TLS 1.2 / 1.3	Identité du serveur, clés de session	`openssl s_client`
Requête	HTTP	La page, un code 200, 301, 404…	`curl -i` , journaux

Chaque étape peut échouer seule : un diagnostic commence par situer **laquelle**.

Apache et Nginx

	Apache HTTP Server	Nginx
Modèle	Processus ou threads (MPM)	Événementiel, peu de processus
Configuration	Hôtes virtuels, `.htaccess` par dossier	Blocs `server`, pas de `.htaccess`
Point fort	Modules, hébergement mutualisé, PHP	Frontal, statique, proxy inverse, charge
Debian : activer, valider	`a2ensite`, `apache2ctl configtest`	`ln -s`, `nginx -t`

Les deux sont libres et parmi les plus déployés au monde, souvent **ensemble**.

Un serveur, plusieurs sites : l'hôte virtuel

Le client envoie le nom demandé dans l'en-tête HTTP `Host`. Le serveur choisit le site qui le déclare.

Requête du client	Site servi par Apache
`Host: web.lab.test`	`lab.conf` (`ServerName web.lab.test`)
`Host: www.lab.test`	`lab.conf` (`ServerAlias www.lab.test`)
`Host: 10.77.1.30`	site par défaut `000-default`

C'est pourquoi `curl --resolve web.lab.test:80:10.77.1.30` teste le **bon** site sans DNS, alors que `curl http://10.77.1.30/` affiche la page Debian.

Un hôte virtuel Apache

```
<VirtualHost *:80>
    ServerName web.lab.test
    ServerAlias www.lab.test
    DocumentRoot /var/www/lab
    CustomLog ${APACHE_LOG_DIR}/lab-access.log combined
</VirtualHost>
```

1. Écrire dans `sites-available/lab.conf`.
2. `a2ensite lab` crée le lien dans `sites-enabled/`.
3. `apache2ctl configtest` **avant** tout rechargement.
4. `systemctl reload apache2`, puis `apache2ctl -S`.

Le proxy inverse : un frontal devant l'application

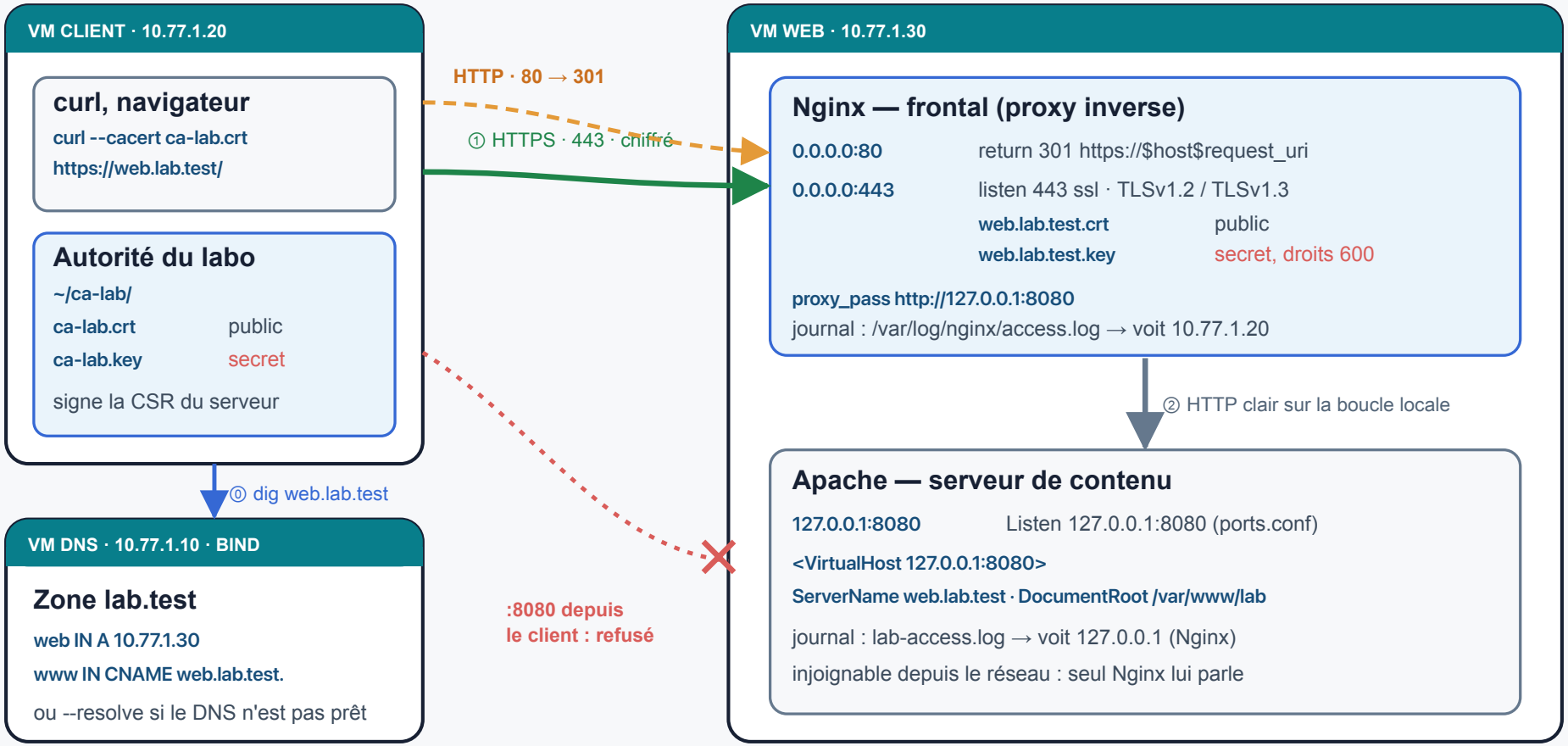
Le client ne parle qu'au frontal ; le frontal transmet au serveur d'application.

Intérêt	Exemple concret
Un seul point d'entrée exposé	Apache, PHP, Node n'écoutent que sur `127.0.0.1`
Terminaison TLS centralisée	Un seul endroit pour les certificats
Répartition de charge	Plusieurs serveurs d'application derrière un frontal
Cache et fichiers statiques	Images et CSS servis sans solliciter l'application
Filtrage	Limitation de débit, pare-feu applicatif (WAF)

Produits courants : Nginx, HAProxy, Traefik, Caddy, Apache (`mod_proxy`), Cloudflare.

L'architecture du laboratoire

Un frontal Nginx devant Apache, et HTTPS de bout en bout côté client



À retenir · TLS protège le trajet client ↔ Nginx. Le certificat lie une clé publique aux noms web/www.lab.test, signé par l'autorité du labo.

ATTENTION

Ce que le frontal change dans les journaux

Journal	Adresse cliente vue	Pourquoi
Nginx `access.log`	`10.77.1.20`	Le client s'est connecté à Nginx
Apache `lab-access.log`	`127.0.0.1`	C'est Nginx qui se connecte à Apache

- Une enquête sur incident qui ne lit qu'Apache **perd l'adresse réelle** du client.
- Correctif : en-têtes `X-Real-IP`, `X-Forwarded-For`, module Apache `remoteip`.
- Ces en-têtes ne sont fiables que **s'ils viennent du frontal** : Apache ne doit pas être joignable directement.

Passer en HTTPS

Ce que HTTPS garantit… et ce qu'il ne garantit pas

Garanti, entre client et Nginx	Non garanti
Confidentialité : contenu chiffré sur le réseau	Que le serveur soit honnête ou non compromis
Intégrité : toute modification est détectée	La sécurité des données stockées sur le serveur
Authentification du serveur : il possède la clé du certificat	La disponibilité du service
	Le chiffrement entre Nginx et Apache

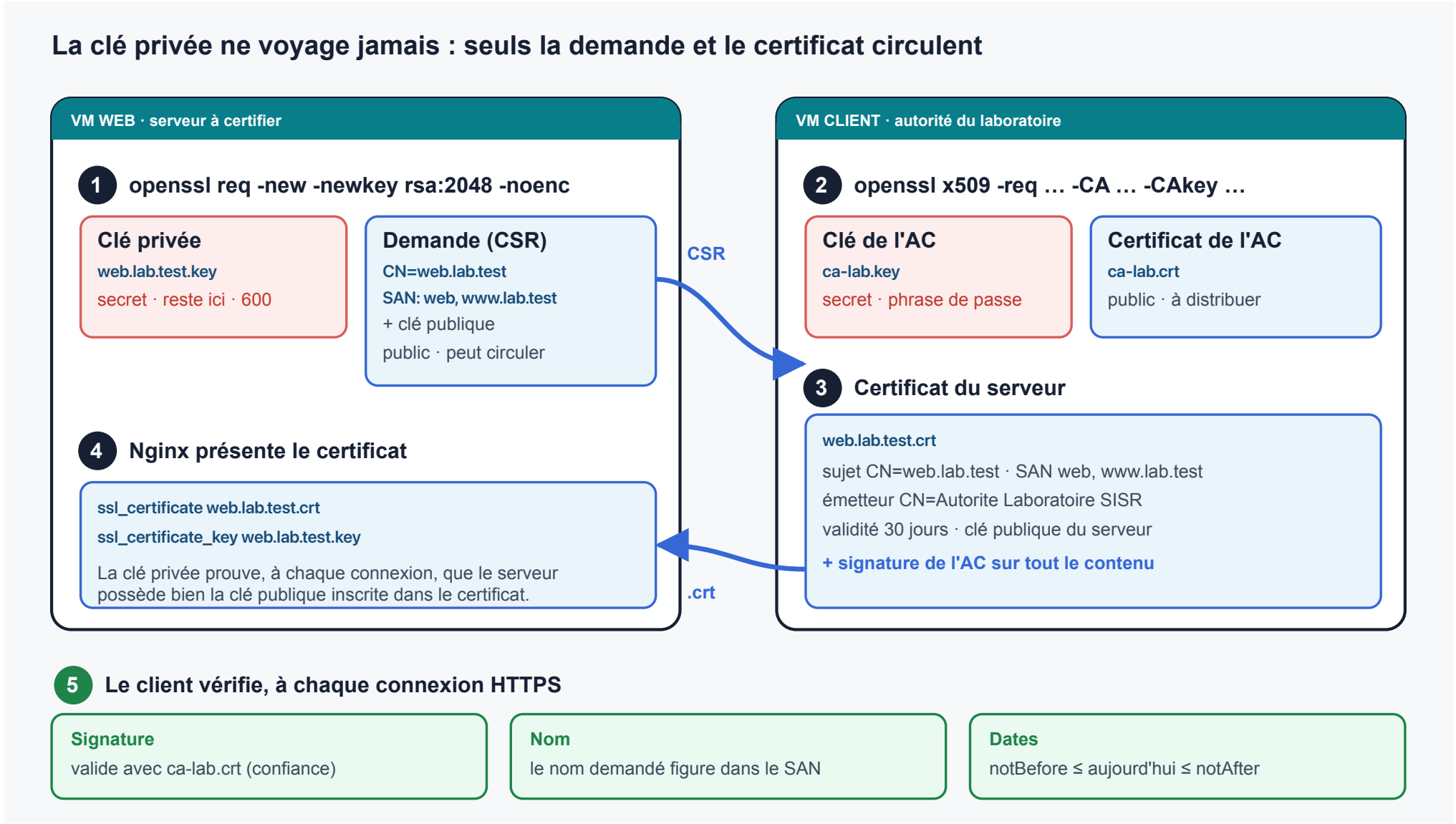
Un cadenas prouve à **qui** l'on parle, pas que cet interlocuteur est **digne de confiance**.

Le certificat X.509

Un certificat **lie une clé publique à des noms**, sous la signature d'une autorité.

Champ	Exemple au laboratoire	Rôle
Sujet	`CN=web.lab.test`	Titulaire (historique)
Noms alternatifs (SAN)	`web.lab.test`, `www.lab.test`	Noms réellement vérifiés
Émetteur	`CN=Autorite Laboratoire SISR`	Qui a signé
Validité	30 jours	Durée d'usage
Clé publique	RSA 2048 bits	Prouvée par la clé privée du serveur
Signature	par `ca-lab.key`	Vérifiable avec `ca-lab.crt`

Obtenir le certificat : la clé privée ne voyage pas



Quelle autorité pour quel usage ?

	AC publique	AC privée d'organisation	Auto-signé
Exemples	Let's Encrypt, DigiCert, Sectigo	Active Directory Certificate Services, step-ca, OpenSSL	`openssl req -x509` sur le serveur
Confiance	Navigateurs, nativement	Postes où l'on déploie le certificat de l'AC	Exception manuelle, poste par poste
Usage	Sites publics	Intranet, laboratoire, VPN, Wi-Fi 802.1X	Test ponctuel uniquement
Coût	Gratuit (Let's Encrypt) à payant	Infrastructure à gérer	Nul, mais habitue à ignorer les alertes

La négociation TLS, simplifiée

Étape	Client	Serveur Nginx
1	`ClientHello` : versions, suites, **nom demandé (SNI)**	
2		`ServerHello` + **certificat** de `web.lab.test`
3	Vérifie signature, nom, dates	Prouve la possession de la clé privée
4	Clés de session partagées	Clés de session partagées
5	Requête HTTP **chiffrée**	Réponse HTTP **chiffrée**

`ssl_protocols TLSv1.2 TLSv1.3` : les versions antérieures, vulnérables, sont refusées.

ATTENTION

Lire une erreur de certificat

Message de curl	Vérification en échec	Premier contrôle
`unable to get local issuer certificate`	Autorité inconnue	`--cacert`, magasin du poste
`no alternative certificate subject name matches`	Nom absent du SAN	`openssl x509 -ext subjectAltName`
`certificate has expired`	Dates	`-dates`, horloge
`Connection refused`	Rien n'écoute sur 443	`ss -ltnp`, `nginx -t`

Ne **jamais** contourner avec `curl -k` dans une preuve : on supprime la vérification.

ACTIVITÉ

TP — Web : Apache, Nginx et HTTPS

Sur les VM · en binôme, conclusion individuelle · lundi et mardi

À produire : tableau commande / attendu / observé / conclusion, configurations, réponses aux questions 1 à 11.

1. Lundi : étapes 1 à 3, Apache en HTTP puis Nginx en frontal.
2. Mardi : étapes 4 et 5, autorité du laboratoire, certificat, HTTPS et redirection.
3. Repli documentaire en fin de fiche si une VM n'est pas prête.

À RETENIR

À retenir

1. L'en-tête `Host` sélectionne l'hôte virtuel : `ServerName` chez Apache, `server_name` chez Nginx.
2. Valider avant de recharger : `apache2ctl configtest`, `nginx -t`.
3. Un frontal expose un seul point d'entrée ; le serveur d'application écoute sur la boucle locale.
4. Un certificat lie une clé publique à des noms, sous la signature d'une autorité de confiance.
5. HTTPS garantit confidentialité, intégrité et identité du serveur sur le trajet, rien de plus.